

EMERGENCY RESOURCE MANAGEMENT SYSTEM

Java Command-Line Project Report

Student: Nayan Vitesh Chavhan

Registration Number: 25BAI10478

Branch: CSE-AIML

VITyarthi - Build Your Own Project

1. Introduction

The Emergency Resource Management System is a simple Java command-line project. It helps a small campus or office team record emergency incidents and manage the resources that can be used to handle them. The main focus is to keep emergency information in one place and make resource assignment simple.

The project uses basic Java concepts such as classes, objects, ArrayList, sorting, file handling, input validation and exception handling. No external Java library is needed.

2. Problem Statement

During an emergency, information can be lost or delayed when incidents and available resources are managed manually. A small digital system can record the incident, give it a priority, show free resources and connect a resource with the incident.

3. Objectives

- Record emergency incidents in a simple way.
- Give each incident HIGH, MEDIUM or LOW priority.
- Store and display available emergency resources.
- Assign an available resource to an incident.
- Update the incident status.
- Show a small summary report.
- Keep the program easy to run from a terminal.

4. Scope

The project is designed for a small campus, office or similar setup. It is a command-line prototype, not a replacement for a real emergency control room. It stores data locally in text files.

5. Target Users

- Campus security or emergency teams
- Office support teams
- Resource coordinators
- Small response teams

6. Functional Requirements

Module	What it does
Incident Management	Adds, displays and stores incidents.
Priority Management	Orders incidents by HIGH, MEDIUM and LOW priority.
Resource Management	Adds, displays and stores emergency resources.
Resource Assignment	Connects a free resource with an incident.
Status Management	Changes an incident to OPEN, ASSIGNED or RESOLVED.

Reports	Shows incident and resource counts.
----------------	-------------------------------------

7. Non-Functional Requirements

Requirement	Project approach
Performance	Uses simple in-memory lists for quick operations.
Usability	Clear numbered terminal menu and simple messages.
Reliability	Data is saved to local files.
Error Handling	Invalid input is checked and friendly error messages are shown.
Maintainability	The code is split into small Java classes.
Resource Efficiency	No heavy framework or external library is used.

8. System Architecture

User → Main Menu → Services → FileStore → Local Data Files

Main controls the menu. IncidentService handles incidents. ResourceService handles resources. ReportService creates the summary. Validator checks user input. FileStore reads and writes the local files.

9. Workflow

Start → Open Menu → Add Resource / Report Incident → Set Priority → Save Data → Assign Resource → Update Status → View Report → Exit

10. Use Case Diagram - Text Form

Actor: Emergency Coordinator

Emergency Coordinator → Report Incident
 Emergency Coordinator → View Incidents
 Emergency Coordinator → Add Resource
 Emergency Coordinator → View Resources
 Emergency Coordinator → Assign Resource
 Emergency Coordinator → Update Incident Status
 Emergency Coordinator → View Report

11. Class Design

Class	Purpose
Main	Shows menu and connects the services.
Incident	Stores incident details.
Resource	Stores resource details.
IncidentService	Handles incident operations.
ResourceService	Handles resource operations.
ReportService	Creates summary information.
FileStore	Reads and writes local files.

Validator	Checks input values.
PriorityManager	Sorts incidents by priority.
IdGenerator	Creates incident and resource IDs.
InputTest	Runs simple validation tests.

12. Sequence: Assigning a Resource

Coordinator enters Incident ID → System checks the incident → System shows available resources → Coordinator selects a resource → System checks that it is free → Resource becomes BUSY → Incident becomes ASSIGNED → Data is saved.

13. Storage Design

The project uses two simple local files. This keeps the project easy to run from the command line.

File	Fields
incidents.txt	id, type, location, priority, status, description, resourceId
resources.txt	id, name, type, status, location

Relationship: one incident can have one assigned resource at a time. A resource can be AVAILABLE or BUSY.

14. Design Decisions and Reason

- Command line was used so the project can run without a GUI.
- ArrayList was used because the number of records is small and it is easy to manage.
- Separate service classes make the code easier to understand.
- Text files were used instead of a database to keep setup simple.
- Priority sorting gives high-priority incidents a clear place in the list.
- Validation prevents empty fields and wrong priority/status values.

15. Implementation Details

The program starts from Main.java. The menu calls the required service. When a new incident or resource is added, an ID is generated and the record is saved. When a resource is assigned, the incident status changes to ASSIGNED and the resource status changes to BUSY. The ReportService counts the current records.

16. Testing Approach

Test	Expected result	Result
HIGH priority	Accepted	Pass
medium priority	Converted to MEDIUM	Pass
LOW priority	Accepted	Pass
Wrong priority	Error shown	Pass
Empty required field	Error shown	Pass

Busy resource assignment	Assignment rejected	Pass
--------------------------	---------------------	------

A small Java test program is included as InputTest.java. It checks valid priority input.

17. Sample Result

Incident: INC001

Type: Medical

Location: Block 4

Priority: HIGH

Status: ASSIGNED

Resource: RES001

18. Challenges Faced

- Keeping incident and resource data connected.
- Making sure a busy resource is not assigned again.
- Handling wrong user input without stopping the program.
- Keeping the project simple enough to run directly from a terminal.

19. Learnings and Key Takeaways

- I learned how to divide a Java project into different classes.
- I learned how ArrayList and sorting can be used in a real problem.
- I learned basic file handling for saving data.
- I learned how validation and exception handling improve a program.
- I learned how to prepare a project for GitHub and command-line execution.

20. Future Enhancements

- Add a proper database.
- Add login for different users.
- Add a GUI or web version.
- Add real-time notifications.
- Add location/map support.
- Add a larger report and response history.

21. References

- VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines.
- Java documentation and standard Java library concepts used in the project.